МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
<БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ>

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №3

Специальность АС-67

Выполнил
Д. А. Ерш,
студент группы АС-67

Проверил
А. А. Крощенко,
ст. преп. кафедры ИИТ,
«___> _____2026 г.

Брест 2026

Цель: На практике сравнить работу нескольких алгоритмов классификации, таких как метод k-ближайших соседей (k-NN), деревья решений и метод опорных векторов (SVM). Научиться подбирать гиперпараметры моделей и оценивать их влияние на результат.

Вариант 4

- `Digits`
- Распознать, какая цифра (от 0 до 9) изображена на картинке 8x8 пикселей
- **Задания:**
  1. Загрузите встроенный в `scikit-learn` набор данных `digits`;
  2. Разделите данные на обучающую и тестовую выборки;
  3. Обучите три модели (k-NN, Decision Tree, SVM) для многоклассовой классификации;
  4. Для каждой модели выведите `classification_report` (sklearn.metrics), содержащий основные метрики для каждого класса;
  5. Сравните общую точность моделей и определите, какая из них лучше всего подходит для этой задачи.

Выполнение:

Код программы

```
import numpy as np
import pandas as pd

from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

print("   Запуск задачи классификации (Digits)")

digits = load_digits()

X = digits.data
y = digits.target

print("Количество записей:", len(X))
print("Количество признаков:", X.shape[1])
print("Количество классов:", len(np.unique(y)))

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Обучающая выборка:", len(X_train))
print("Тестовая выборка:", len(X_test))

models = {
    "k-NN": make_pipeline(
        StandardScaler(),
        KNeighborsClassifier(n_neighbors=5)
    ),
    "Decision Tree": DecisionTreeClassifier(
        random_state=42
    ),
    "SVM": make_pipeline(
```

```
        StandardScaler(),
        SVC(kernel="rbf", C=1.0, gamma="scale")
    )
}

results = []

for name, model in models.items():

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)

    results.append({
        "Модель": name,
        "Accuracy": accuracy
    })

    print(f"\n{name}")
    print(f"Accuracy: {accuracy:.4f}")

    print("\nClassification Report:")
    print(classification_report(
        y_test,
        y_pred,
        labels=np.arange(10),
        target_names=[str(i) for i in range(10)],
        digits=4
    ))

results = pd.DataFrame(results).sort_values(
    by="Accuracy",
    ascending=False
)

print("\nСРАВНЕНИЕ МОДЕЛЕЙ")
print(results.to_string(
    index=False,
    float_format=lambda x: f"{x:.4f}"
))

print("\nЛУЧШАЯ МОДЕЛЬ")
print(results.iloc[0]["Модель"])
print(f"Accuracy: {results.iloc[0]['Accuracy']:.4f}")
```

Результат:

Запуск задачи классификации (Digits)
Количество записей: 1797
Количество признаков: 64
Количество классов: 10
Обучающая выборка: 1437
Тестовая выборка: 360

k-NN
Accuracy: 0.9639

Classification Report:

	precision	recall	f1-score	support
0	1.0000	1.0000	1.0000	36
1	0.9231	1.0000	0.9600	36
2	0.9459	1.0000	0.9722	35
3	1.0000	0.9730	0.9863	37
4	0.9714	0.9444	0.9577	36

5	0.9730	0.9730	0.9730	37
6	0.9730	1.0000	0.9863	36
7	0.9211	0.9722	0.9459	36
8	0.9688	0.8857	0.9254	35
9	0.9697	0.8889	0.9275	36
accuracy			0.9639	360
macro avg	0.9646	0.9637	0.9634	360
weighted avg	0.9648	0.9639	0.9636	360

Decision Tree
Accuracy: 0.8250

Classification Report:

	precision	recall	f1-score	support
0	0.9211	0.9722	0.9459	36
1	0.8387	0.7222	0.7761	36
2	0.7778	0.8000	0.7887	35
3	0.7895	0.8108	0.8000	37
4	0.8611	0.8611	0.8611	36
5	0.8750	0.9459	0.9091	37
6	0.8108	0.8333	0.8219	36
7	0.8378	0.8611	0.8493	36
8	0.7059	0.6857	0.6957	35
9	0.8182	0.7500	0.7826	36
accuracy			0.8250	360
macro avg	0.8236	0.8242	0.8230	360
weighted avg	0.8241	0.8250	0.8237	360

SVM
Accuracy: 0.9750

Classification Report:

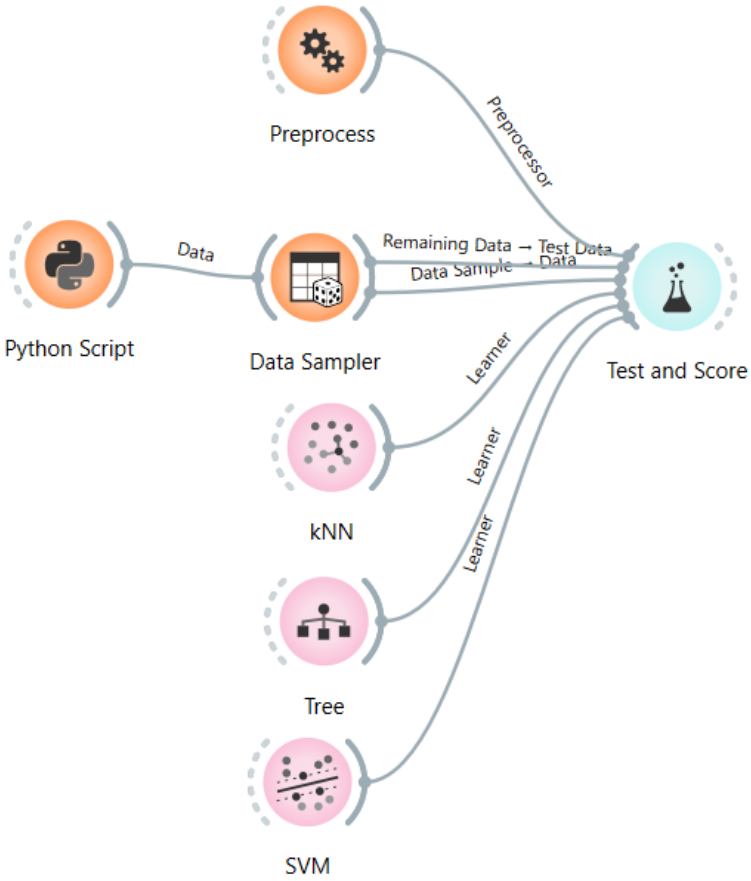
	precision	recall	f1-score	support
0	1.0000	1.0000	1.0000	36
1	0.9459	0.9722	0.9589	36
2	1.0000	1.0000	1.0000	35
3	1.0000	1.0000	1.0000	37
4	0.9459	0.9722	0.9589	36
5	0.9737	1.0000	0.9867	37
6	0.9730	1.0000	0.9863	36
7	0.9211	0.9722	0.9459	36
8	1.0000	0.9143	0.9552	35
9	1.0000	0.9167	0.9565	36
accuracy			0.9750	360
macro avg	0.9760	0.9748	0.9748	360
weighted avg	0.9759	0.9750	0.9749	360

СРАВНЕНИЕ МОДЕЛЕЙ

Модель	Accuracy
SVM	0.9750
k-NN	0.9639
Decision Tree	0.8250

ЛУЧШАЯ МОДЕЛЬ
SVM
Accuracy: 0.9750

Результаты в Orange:



Evaluation results for target (None, show average over classes)

Model	AUC	CA	F1	Prec	Recall	MCC
kNN	0.997	0.978	0.978	0.978	0.978	0.975
Tree	0.929	0.864	0.864	0.869	0.864	0.849
SVM	1.000	0.986	0.986	0.986	0.986	0.985

Вывод: В ходе лабораторной работы была выполнена многоклассовая классификация изображений рукописных цифр от 0 до 9 с использованием набора данных Digits. Были обучены и сравнены три алгоритма машинного обучения: k-NN, Decision Tree и SVM. Наилучший результат показала модель SVM с точностью 97,50%, далее следует k-NN с результатом 96,39%. Наименьшую точность показало дерево решений — 82,50%. Таким образом, для распознавания рукописных цифр среди рассмотренных алгоритмов наиболее эффективным оказался метод опорных векторов SVM.